

베이지안 기계학습

5. Bayesian inference for regression models II

이경재

Table of Contents

- ▶ Bayesian inference for high-dimensional linear regression models
 - horseshoe prior
- ▶ Bayesian inference for high-dimensional generalized linear regression models
 - Logistic regression
- ▶ Remark and next week

Table of Contents

- ▶ Bayesian inference for high-dimensional linear regression models
 - horseshoe prior
- ▶ Bayesian inference for high-dimensional generalized linear regression models
 - Logistic regression
- ▶ Remark and next week

Bayesian inference for high-dimensional linear regression models

High-dimensional regression

$$Y_i = X_i^T \beta + \epsilon_i, \quad \epsilon_i \stackrel{iid}{\sim} N(0, \sigma^2), \quad i = 1, \dots, n.$$

- ▶ (High-dimensional setting) $p > n$ or even $p \gg n$
- ▶ For consistency, we need to reduce the number of parameters by assuming that the true coefficient vector $\beta_0 \in \mathbb{R}^p$ is **sparse**, i.e., $\|\beta_0\|_0 = \sum_{j=1}^p I(\beta_{0j} \neq 0) = s_0 \ll p$.

Motivations for shrinkage priors

- ▶ There is a **spike and slab prior** which assign nonzero or exact zero for each coefficient β_j . It is definitely a natural choice for estimating a sparse coefficient vector.
- ▶ However, by using spike and slab priors, we should explore the model space which consists of 2^p models.
- ▶ Even when $p = 100$, the model space contains $2^p > 10^{30}$ models, which is too large to be fully explored.
- ▶ This problem occurs because when using a spike and slab prior, we have to determine whether each β_j is 0 or not.

Global-local shrinkage priors

$$\begin{aligned}\beta_j \mid \lambda_j, \tau, \sigma^2 &\stackrel{ind}{\sim} N(0, \lambda_j^2 \tau^2 \sigma^2), \\ \lambda_j &\stackrel{iid}{\sim} \pi_L, \quad j = 1, \dots, p, \\ \tau &\sim \pi_G.\end{aligned}$$

- ▶ Let's consider the above Gaussian scale mixture priors, which leads to a continuous distribution for β_j .
- ▶ We don't need to determine whether each β_j is 0 or not.
- ▶ It is called a **global-local** shrinkage prior.

Global-local shrinkage priors

$$\begin{aligned}\beta_j \mid \lambda_j, \tau, \sigma^2 &\stackrel{ind}{\sim} N(0, \lambda_j^2 \tau^2 \sigma^2), \\ \lambda_j &\stackrel{iid}{\sim} \pi_L, \quad j = 1, \dots, p, \\ \tau &\sim \pi_G.\end{aligned}$$

- ▶ (Global shrinkage parameter)
 τ^2 controls the overall shrinkage effect.

(Local shrinkage parameter)

λ_j^2 governs the shrinkage level of individual parameter to avoid over-shrinking nonzero signals.

Global-local shrinkage priors

$$\begin{aligned}\beta_j \mid \lambda_j, \tau, \sigma^2 &\stackrel{\text{ind}}{\sim} N(0, \lambda_j^2 \tau^2 \sigma^2), \\ \lambda_j &\stackrel{\text{iid}}{\sim} \pi_L, \quad j = 1, \dots, p, \\ \tau &\sim \pi_G.\end{aligned}$$

- ▶ (Global shrinkage parameter)
 τ^2 controls the overall shrinkage effect.
- ▶ (Local shrinkage parameter)
 λ_j^2 governs the shrinkage level of individual parameter to avoid over-shrinking nonzero signals.

*Various global-local shrinkage priors

$$\beta_j \mid \lambda_j, \tau, \sigma^2 \stackrel{\text{ind}}{\sim} N(0, \lambda_j^2 \tau^2 \sigma^2)$$

	Prior on λ_j^2	Prior on τ^2
Laplace	Exponential	Fixed
Student- t	Inverse-gamma	Fixed
Horseshoe ¹	Half-Cauchy	Half-Cauchy

$$\beta_j \mid \lambda_j, \tau, \sigma^2 \stackrel{\text{ind}}{\sim} N(0, \lambda_j^2 \phi_j^2 \tau^2 \sigma^2)$$

	Prior on λ_j^2	Prior on ϕ_j^2	Prior on τ^2
Generalized double Pareto ²	Exponential	Gamma	Fixed
Dirichlet-Laplace ³	Exponential	$(\phi_1, \dots, \phi_p) \sim$ Dirichlet	$\tau \sim$ Gamma

¹ Carvalho et al. (2010) The horseshoe estimator for sparse signals.

² Armagan et al. (2013) Generalized double pareto shrinkage.

³ Bhattacharya et al. (2015) Dirichlet-Laplace priors for optimal shrinkage.

Horseshoe prior

$$\begin{aligned}\beta_j \mid \lambda_j, \tau, \sigma^2 &\stackrel{\text{ind}}{\sim} N(0, \lambda_j^2 \tau^2 \sigma^2), \\ \lambda_j &\stackrel{\text{iid}}{\sim} C^+(0, 1), \quad j = 1, \dots, p, \\ \tau &\sim C^+(0, 1).\end{aligned}$$

- ▶ One of the most popular shrinkage priors is the above **horseshoe** prior.
- ▶ $C^+(0, 1)$ (half-Cauchy) has the pdf $f(x) = \frac{2}{\pi}(1+x^2)^{-1}$, $x > 0$.
- ▶ For σ^2 , we will use the Jeffreys' prior: $\pi(\sigma^2) \propto 1/\sigma^2$.

Horseshoe prior: pdf

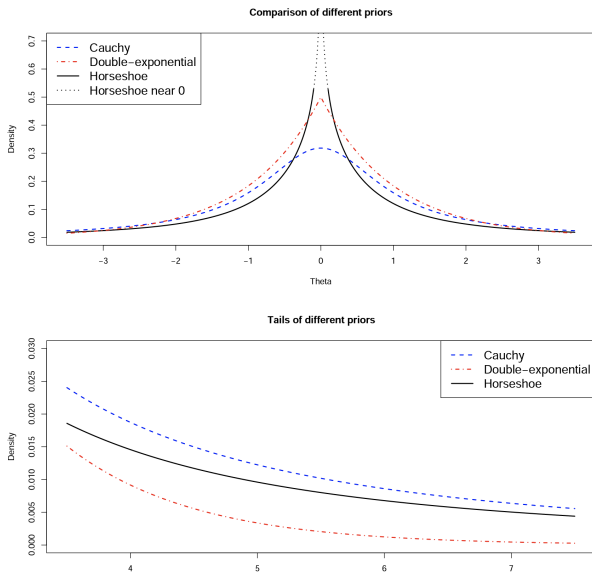
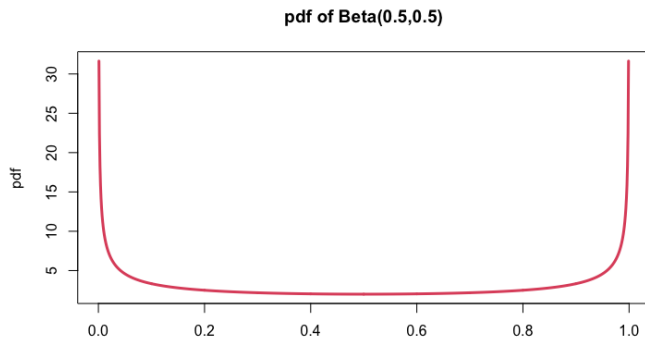


Figure: Horseshoe and other heavy tail distributions (from Carvalho et al. (2010) The horseshoe estimator for sparse signals).

Why do we call it horseshoe?



- ▶ If $\lambda_j \sim C^+(0, 1)$, then $\kappa_j := (1 + \lambda_j^2)^{-1} \sim \text{Beta}(0.5, 0.5)$.
- ▶ Note that $\lambda_j^2 = (1 - \kappa_j)/\kappa_j$ and

$$\beta_j \mid \lambda_j, \tau, \sigma^2 \stackrel{\text{ind}}{\sim} N\left(0, \frac{1 - \kappa_j}{\kappa_j} \tau^2 \sigma^2\right).$$

Horseshoe prior

- ▶ The induced posterior has nice theoretical properties in various settings.
 - Linear regression⁴
 - Covariance matrix⁵
 - Deep neural network⁶
- ▶ Although the horseshoe prior enables efficient model space exploration, theoretical properties of the posterior have tended to be clarified more slowly than the spike and slab prior.

⁴ Song and Liang (2023) Nearly optimal Bayesian shrinkage for high-dimensional regression.

⁵ Lee et al. (2022) The beta-mixture shrinkage prior for sparse covariances with near-minimax posterior convergence rate.

⁶ Lee and Lee (2022) Asymptotic properties for Bayesian neural network in Besov space.

Toy example

- ▶ Consider

$$y_{j,l} = \beta_j + \epsilon_{j,l}$$

with $\epsilon_{j,l} \stackrel{iid}{\sim} N(0, \sigma^2)$ for $j = 1, \dots, 1000$ and $l = 1, 2$.⁷

- ▶ Let $\beta_1 = \dots = \beta_{10} = 10$, $\beta_{11} = \dots = \beta_{100} = 2$ and $\beta_j = 0$ otherwise.
- ▶ Compare two priors for β_j 's:
 1. Laplace prior,
 2. horseshoe prior.

⁷Carvalho et al. (2009) Handling sparsity via the horseshoe.

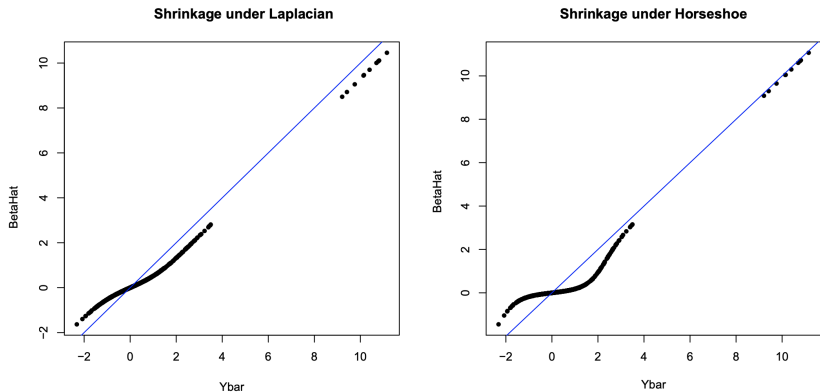


Figure: $\bar{y}_i$ vs. $\hat{\beta}_j$ for Laplace (left) and horseshoe (right) priors. The diagonal lines are where $\hat{\beta}_j = \bar{y}_i$ (from Carvalho et al. (2009)).

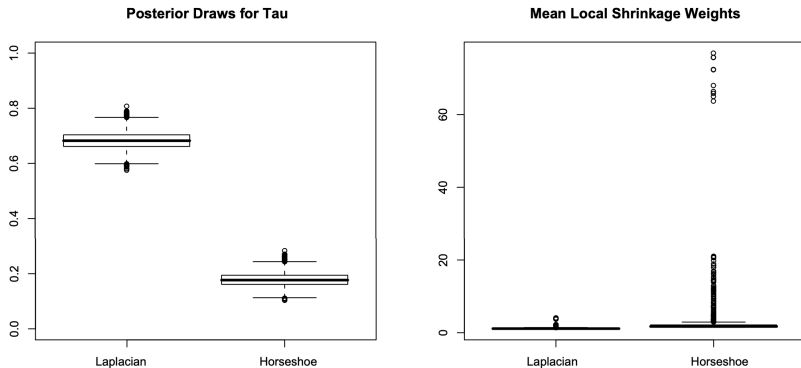


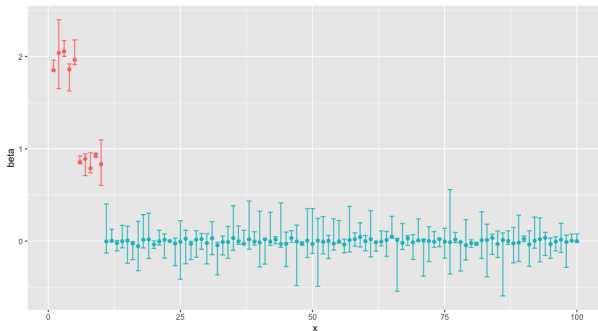
Figure: Posterior samples of τ and λ_j for Laplace (left) and horseshoe (right) priors (from Carvalho et al. (2009)).

Posterior computation

$$\pi(\beta, \lambda_j, \tau, \sigma^2 | y) \propto f(y | \beta, \sigma^2) \prod_{j=1}^p \pi(\beta_j | \lambda_j, \tau, \sigma^2) \pi(\lambda_j) \pi(\tau) \pi(\sigma^2)$$

- ▶ We can derive a Gibbs sampler for posterior inference.
- ▶ However, in this lecture, we just let [rstan](#) do that and won't go into the details.

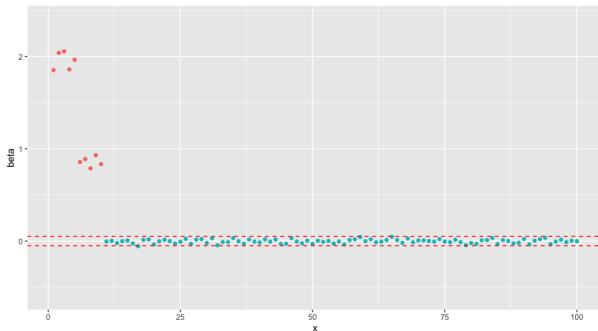
Remark: variable selection



► Variable selection with horseshoe priors

- Credible intervals
- Thresholding approach
- 2-means clustering

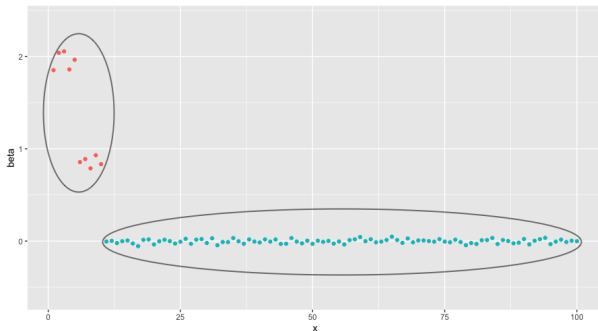
Remark: variable selection



- ▶ Variable selection with horseshoe priors
 - Credible intervals
 - **Thresholding approach**⁸
 - 2-means clustering

⁸ Carvalho et al. (2010) The horseshoe estimator for sparse signals.

Remark: variable selection



► Variable selection with horseshoe priors

- Credible intervals
- Thresholding approach
- 2-means clustering⁹

⁹ Li and Pati (2017) Variable selection using shrinkage priors.

R code: simulation data

```
library(rstan)

n = 100
p = 150

set.seed(1)
X = matrix(rnorm(n*p), nrow=n, ncol=p)
beta.true = c(rep(1,5), rep(0, p-5))
sig2.true = 1
y = rnorm(n, mean=X%*%beta.true, sd=sqrt(sig2.true))
```

R code

```
horseshoe.reg <- "  
  data {  
    int<lower=0> n;           // number of observations  
    int<lower=0> p;           // number of predictors  
    vector[n] y;             // outputs  
    matrix[n,p] X;           // inputs  
  }  
  
  parameters {  
    vector[p] z;             // non-centered parametrization  
    real<lower=0> tau;        // global shrinkage  
    vector<lower=0>[p] lambda; // local shrinkage  
    real<lower=0> sigma2;  
  }
```

R code

```
transformed parameters {  
  vector[p] beta;  
  // implies  $\beta \sim \text{normal}(0, \lambda * \tau)$   
  beta = z .* lambda * tau;  
}  
  
model {  
  z ~ std_normal();  
  lambda ~ student_t(1, 0, 1);  
  tau ~ student_t(1, 0, 1);  
  target += -log(sigma2);  
  y ~ normal(X*beta, sqrt(sigma2));  
}
```

"

R code: remark

- ▶ In this code, we use the following reparametrization:

$$\begin{aligned}\beta_j &\stackrel{d}{=} z_j \cdot \lambda_j \cdot \tau, \\ z_j &\stackrel{iid}{\sim} N(0, 1), \quad j = 1, \dots, p,\end{aligned}$$

which is equivalent to

$$\beta_j \mid \lambda_j, \tau \stackrel{ind}{\sim} N(0, \lambda_j^2 \tau^2), \quad j = 1, \dots, p.$$

- ▶ While this step is not essential, it can be useful to reduce correlations among parameters, especially when the data size is small.¹⁰

¹⁰Betancourt, Michael, and Mark Girolami (2013) Hamiltonian Monte Carlo for Hierarchical Models. <https://arxiv.org/abs/1312.0906>

R code

```
stanC <- stan_model(model_code = horseshoe.reg)
stanD <- list(y = y, X = X, n = n, p = p)
stanF <- sampling(object = stanC, data = stanD,
  chain = 3, iter = 1000,
  warmup = 200, thin = 1)
```

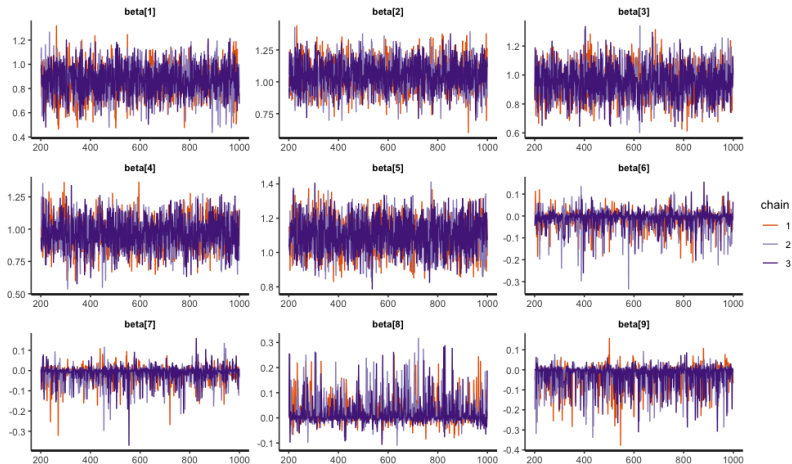
R code

```
beta.1to9 = c("beta[1]", "beta[2]", "beta[3]",  
              "beta[4]", "beta[5]", "beta[6]",  
              "beta[7]", "beta[8]", "beta[9]")  
print(stanF, pars = beta.1to9)
```

	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
beta[1]	0.86	0	0.13	0.61	0.78	0.87	0.95	1.11	2040	1
beta[2]	1.05	0	0.12	0.82	0.97	1.05	1.13	1.28	1954	1
beta[3]	0.95	0	0.11	0.74	0.88	0.95	1.02	1.17	2025	1
beta[4]	0.96	0	0.12	0.73	0.89	0.96	1.04	1.19	2210	1
beta[5]	1.10	0	0.10	0.90	1.03	1.10	1.16	1.29	1972	1
beta[6]	-0.01	0	0.04	-0.12	-0.01	0.00	0.00	0.05	1544	1
beta[7]	-0.01	0	0.04	-0.12	-0.02	0.00	0.00	0.04	2376	1
beta[8]	0.02	0	0.05	-0.04	0.00	0.00	0.02	0.16	1641	1
beta[9]	-0.02	0	0.05	-0.18	-0.03	0.00	0.00	0.04	1904	1

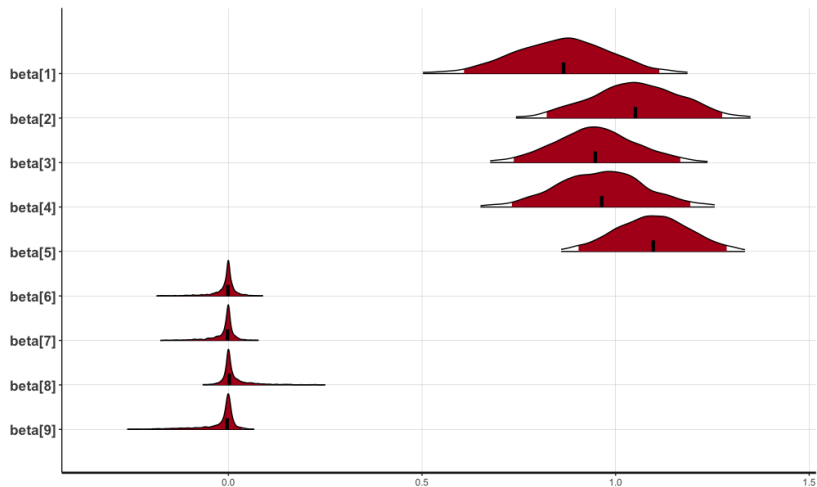
R code

```
stan_trace(stanF, pars= beta.1to9)
```



R code

```
plot(stanF, show_density = T, ci_level = 0.95,  
     pars = beta.1to9, outer_level = 0.99)
```



R code: variable selection

```
# post.samples <- extract(stanF)
# beta.samples <- post.samples$beta

beta.samples <- as.matrix(stanF, pars = "beta")
beta.CIs <- apply(beta.samples, 2,
                  quantile, c(0.025, 0.975))

beta.CIs[,1:10]
```

```
> beta.CIs[,1:10]
  parameters
    beta[1]  beta[2]  beta[3]  beta[4]  beta[5]  beta[6]  beta[7]
2.5%  0.6085107 0.8220223 0.7369572 0.7321654 0.9042674 -0.12227679 -0.11713913
97.5% 1.1127898 1.2757743 1.1669512 1.1928557 1.2867948  0.04677017  0.03873058
  parameters
    beta[8]  beta[9]  beta[10]
2.5% -0.03505764 -0.1812800 -0.12033909
97.5%  0.15617554  0.0350385  0.04368895
```

R code: variable selection

```
beta.var.sel <- rep(0, p)
for(i in 1:p){
  beta.var.sel[i] =
    (beta.CIs[1,i] > 0 | beta.CIs[2,i] < 0)
}
```

```
which(beta.var.sel == 1)
```

```
> which(beta.var.sel == 1)
[1] 1 2 3 4 5
```

Table of Contents

- ▶ Bayesian inference for high-dimensional linear regression models
 - horseshoe prior
- ▶ Bayesian inference for high-dimensional generalized linear regression models
 - Logistic regression
- ▶ Remark and next week

Bayesian inference for high-dimensional generalized linear regression models

Logistic regression

$$Y_i \mid \beta \stackrel{ind}{\sim} \text{Ber}(p_i),$$
$$\log\left(\frac{p_i}{1-p_i}\right) = \alpha + X_i^T \beta, \quad i = 1, \dots, n.$$

- ▶ $Y_i \in \{0, 1\}$: binary response variable
- ▶ $X_i \in \mathbb{R}^p$: covariate vector
- ▶ As before, we can apply the horseshoe prior for β .
- ▶ Again, [rstan](#) will conduct the posterior inference.

R code: Default data

```
library(rstan)
library(ISLR) # to use Default data

y = as.numeric(Default$default) - 1
n = nrow(Default)
p = 100
X = matrix(1, nrow=n, ncol=p)
X[,1] = as.numeric(Default$student) - 1
X[,2] = Default$balance
X[,3] = Default$income
X[,2:3] = scale(X[,2:3])

// fake variables
X[,4:p] = matrix(rnorm(n*(p-3)), nrow=n, ncol=p-3)
```

R code

```
horseshoe.logit.reg <- "  
  data {  
    int<lower=0> n;  
    int<lower=0> p;  
    int<lower=0,upper=1> y[n];  
    matrix[n,p] X;  
  }  
  
  parameters {  
    vector[p] z;  
    real<lower=0> tau;  
    vector<lower=0>[p] lambda;  
    real<lower=0> sigma2;  
  }
```

R code

```
transformed parameters {  
  vector[p] beta;  
  beta = z .* lambda * tau;  
}
```

```
model {  
  z ~ std_normal();  
  lambda ~ student_t(1, 0, 1);  
  tau ~ student_t(1, 0, 1);  
  target += -log(sigma2);  
  y ~ bernoulli_logit_glm(X, 0, beta);  
}
```

"

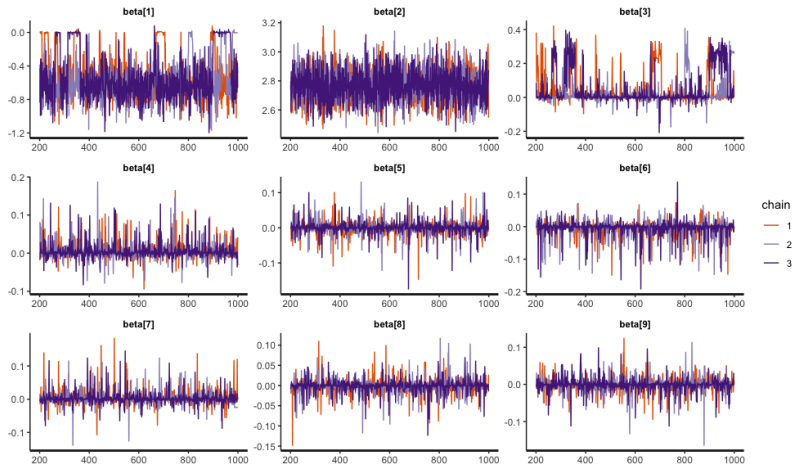
R code

```
beta.1to9 = c("beta[1]", "beta[2]", "beta[3]",  
              "beta[4]", "beta[5]", "beta[6]",  
              "beta[7]", "beta[8]", "beta[9]")  
print(stanF, pars = c("alpha", beta.1to9))
```

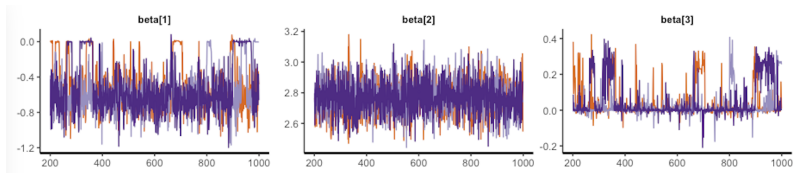
	mean	se_mean	sd	2.5%	25%	50%	75%	97.5%	n_eff	Rhat
alpha	-6.03	0.01	0.19	-6.42	-6.16	-6.02	-5.89	-5.66	1077	1.00
beta[1]	-0.54	0.03	0.27	-0.96	-0.72	-0.60	-0.44	0.00	107	1.01
beta[2]	2.77	0.00	0.11	2.56	2.69	2.77	2.84	2.99	2562	1.00
beta[3]	0.04	0.01	0.08	-0.03	0.00	0.00	0.02	0.29	81	1.02
beta[4]	0.00	0.00	0.02	-0.02	0.00	0.00	0.01	0.06	1931	1.00
beta[5]	0.00	0.00	0.02	-0.04	0.00	0.00	0.00	0.04	2138	1.00
beta[6]	0.00	0.00	0.02	-0.07	-0.01	0.00	0.00	0.03	1765	1.00
beta[7]	0.00	0.00	0.02	-0.03	0.00	0.00	0.00	0.05	2133	1.00
beta[8]	0.00	0.00	0.02	-0.04	0.00	0.00	0.00	0.03	2232	1.00
beta[9]	0.00	0.00	0.02	-0.04	0.00	0.00	0.00	0.04	1966	1.00

R code

```
stan_trace(stanF, pars=beta.1to9)
```



R code: remark



- ▶ In fact, traceplots for β_1 and β_3 imply that we need **longer MCMC chains**.
- ▶ However, for the sake of time, let's proceed with the posterior inference using the MCMC samples obtained so far.

R code

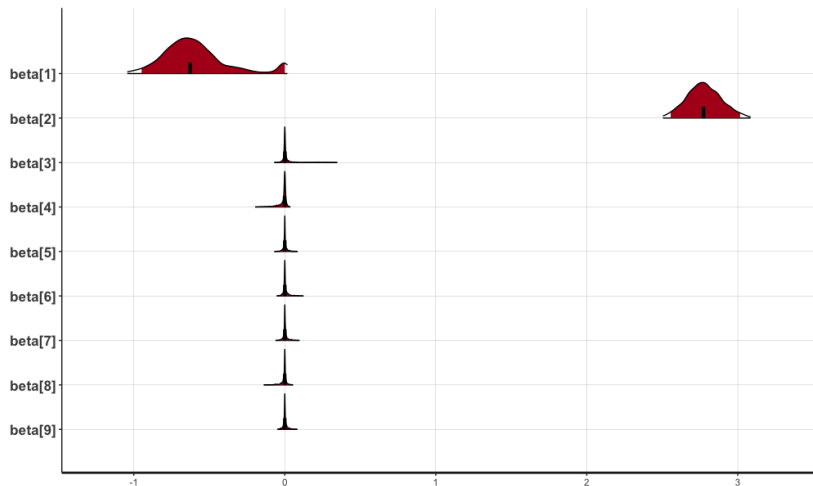
```
beta.samples <- as.matrix(stanF, pars = "beta")
beta.CIs <- apply(beta.samples, 2,
                  quantile, c(0.025, 0.975))

beta.var.sel <- rep(0, p)
for(i in 1:p){
  beta.var.sel[i] =
    (beta.CIs[1,i] > 0 | beta.CIs[2,i] < 0)
}
which(beta.var.sel == 1)

> which(beta.var.sel == 1)
[1] 1 2
```

R code

```
plot(stanF, show_density = T, ci_level = 0.95,  
pars = beta.1to9, outer_level = 0.99)
```



R code

```
post.samples <- extract(stanF)
alpha.samples <- post.samples$alpha
beta.samples <- post.samples$beta

fitted.prob <- rep(0, n)
inv.logit <- function(x) 1/(1+exp(-x))
for(i in 1:n){
  eta = inv.logit(alpha.samples +
    as.vector(beta.samples%*%as.matrix(X[i,])))
  fitted.prob[i] <- mean(eta)
}
```

R code

```
pred.val = ifelse(fitted.prob > 0.5, 1, 0)
table(y, pred.val)
```

```
> table(y, pred.val)
      pred.val
y          0      1
0 9630 37
1 226 107
```

Table of Contents

- ▶ Bayesian inference for high-dimensional linear regression models
 - horseshoe prior
- ▶ Bayesian inference for high-dimensional generalized linear regression models
 - Logistic regression
- ▶ Remark and next week

Remark and next week

Remark

- ▶ In high-dimensional regression models, the use of priors like Zellner's g -prior, which might be suitable in low-dimensional settings, is known to be either impossible or unsatisfactory in terms of performance.
- ▶ In today's lecture, we introduced one alternative, the horseshoe prior, and practiced posterior inference.
- ▶ Although not covered in detail today, another alternative is the **spike and slab prior**, which is also an important and commonly used prior.

Next week

Bayesian network